

Practical – 8

AIM: Prepare the implementation view: Draw Component diagram and Deployment diagram.

- **Objectives:** to explore and use various elements UML component diagram and deployment diagram.

- **Theory:**

- A component diagram is used to break down a large object-oriented system into the smaller components, so as to make them more manageable. It models the physical view of a system such as executables, files, libraries, etc. that resides within the node.
- It visualizes the relationships as well as the organization between the components present in the system. It helps in forming an executable system. A component is a single unit of the system, which is replaceable and executable. The implementation details of a component are hidden, and it necessitates an interface to execute a function. It is like a black box whose behavior is explained by the provided and required interfaces.

- **Purpose of a Component Diagram**

Since it is a special kind of a UML diagram, it holds distinct purposes. It describes all the individual components that are used to make the functionalities, but not the functionalities of the system. It visualizes the physical components inside the system. The components can be a library, packages, files, etc.

The component diagram also describes the static view of a system, which includes the organization of components at a particular instant. The collection of component diagrams represents a whole system.

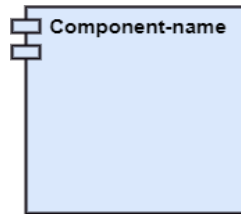
The main purpose of the component diagram are enlisted below:

1. It envisions each component of a system.
2. It constructs the executable by incorporating forward and reverse engineering.
3. It depicts the relationships and organization of components.

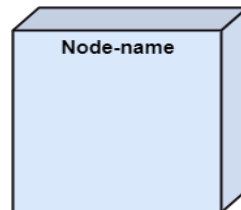
- **Background / Preparation:**

- **Notation of a Component Diagram**

- a) A component



b) A node



○ When to use a Component Diagram?

It represents various physical components of a system at runtime. It is helpful in visualizing the structure and the organization of a system. It describes how individual components can together form a single system. Following are some reasons, which tells when to use component diagram:

1. To divide a single system into multiple components according to the functionality.
2. To represent the component organization of the system.

● Tools / Material Needed:

- Hardware:
- Software:

● Procedure / Steps:

○ How to Draw a Component Diagram?

- The component diagram is helpful in representing the physical aspects of a system, which are files, executables, libraries, etc. The main purpose of a component diagram is different from that of other diagrams. It is utilized in the implementation phase of any application.
- Once the system is designed employing different UML diagrams, and the artifacts are prepared, the component diagram is used to get an idea of implementation. It plays an essential role in implementing applications efficiently.

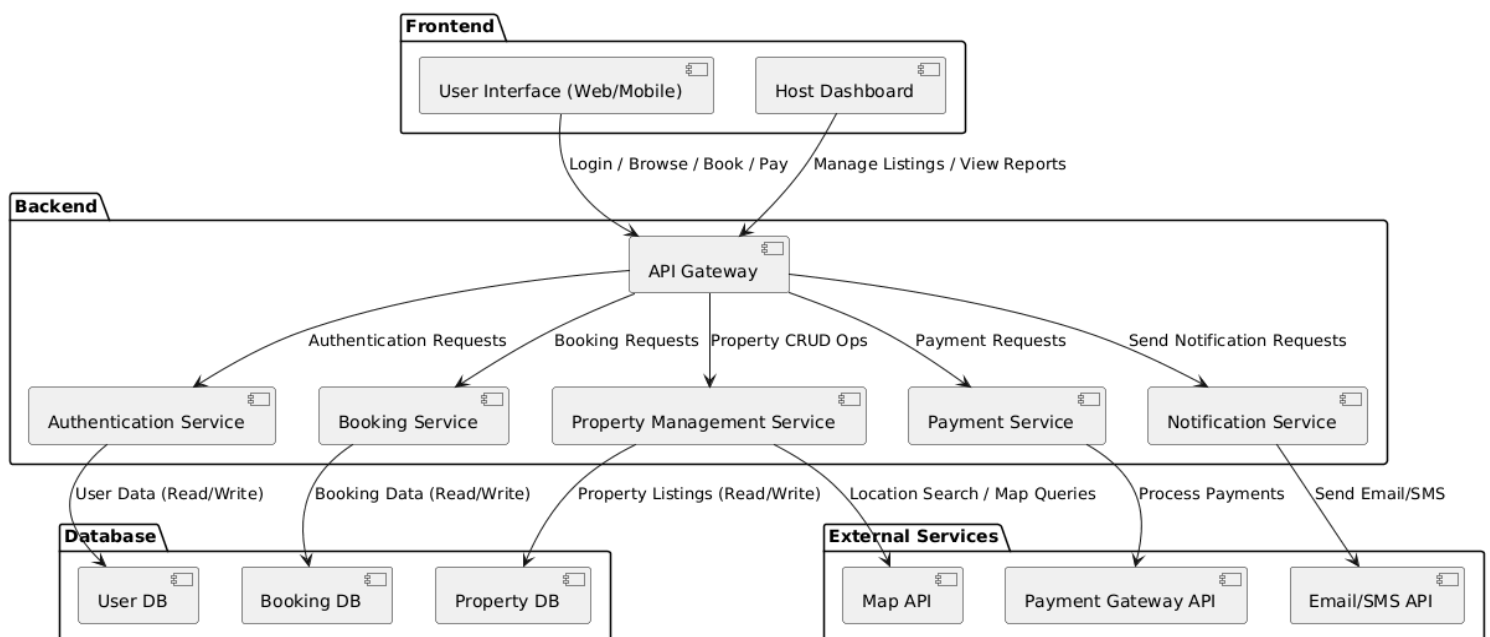
- Following are some artifacts that are needed to be identified before drawing a component diagram:
 1. What files are used inside the system?
 2. What is the application of relevant libraries and artifacts?
 3. What is the relationship between the artifacts?

- Following are some points that are needed to be kept in mind after the artifacts are identified:
 1. Using a meaningful name to ascertain the component for which the diagram is about to be drawn.
 2. Before producing the required tools, a mental layout is to be made.
 3. To clarify the important points, notes can be incorporated.

○ Example of a Component Diagram

- The component diagram for the **Airbnb system** illustrates the high-level organization of the application into **frontend**, **backend services**, **databases**, and **external APIs**. It focuses on the structural view of the system that supports key Airbnb functionalities such as property listing, booking, authentication, payments, and notifications. The diagram reflects how various software components interact to deliver a seamless experience to both hosts and guests.

Component Diagram with Data Flows - Airbnb System



In the **Frontend**, two main dashboards are defined:

- The **User Interface (Web/Mobile App)**, which allows guests to **register, log in, search for properties, make bookings, process payments, and view booking history**.
- The **Host Dashboard**, which enables property owners to **add, update, or remove property listings, view reports, and manage their bookings**.

All frontend requests are routed through the **API Gateway** in the **Backend**, which acts as the **central entry point** for all user and host operations.

The backend is composed of several independent microservices, each responsible for a specific domain of functionality:

- The **Authentication Service** manages **user login, registration, and session validation** by interacting with the **User DB**.
- The **Property Management Service** handles **CRUD operations** for property listings, supported by the **Property DB**.
- The **Booking Service** processes **reservation requests, manages availability, and records booking details**, interacting with the **Booking DB**.
- The **Payment Service** securely handles **payment transactions** by integrating with the external **Payment Gateway API**.
- The **Notification Service** is responsible for sending **booking confirmations, alerts, and updates** via the external **Email/SMS API**.

Additionally, the backend integrates with a **Map API**, which provides **location-based search and mapping functionality** to enhance the property browsing experience.

The **Database Layer** includes three separate data stores to ensure modular and efficient data management:

- **User DB** stores user profiles, authentication credentials, and account preferences.
- **Property DB** stores property details such as location, pricing, availability, and host information.
- **Booking DB** stores booking transactions, stay details, and payment references.

This component diagram demonstrates how all parts of the **Airbnb system** interact cohesively: the **frontend dashboards** send user requests to the **API Gateway**, which directs them to appropriate

backend services. These services communicate with the **databases** for persistent data storage and utilize **external APIs** for payments, notifications, and mapping features.

Overall, the diagram provides a **clear visualization of the static structure** of the Airbnb application, showing how components are organized, connected, and communicate with one another. It effectively depicts the **implementation view** of the system and supports a modular, service-oriented architecture designed for scalability, maintainability, and secure operation.

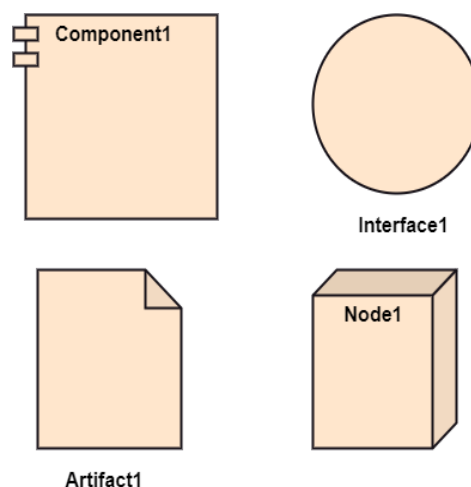
○ Purpose of Deployment Diagram

- The main purpose of the deployment diagram is to represent how software is installed on the hardware component. It depicts in what manner a software interacts with hardware to perform its execution.
- Both the deployment diagram and the component diagram are closely interrelated to each other as they focus on software and hardware components. The component diagram represents the components of a system, whereas the deployment diagram describes how they are actually deployed on the hardware.
- The deployment diagram does not focus on the logical components of the system, but it put its attention on the hardware topology.
- Following are the purposes of deployment diagram enlisted below:
 - To envision the hardware topology of the system.
 - To represent the hardware components on which the software components are installed.
 - To describe the processing of nodes at the runtime.

○ Symbol and notation of Deployment diagram

- The deployment diagram consist of the following notations:

1. A component
2. An artifact
3. An interface
4. A node



○ How to draw a Deployment Diagram?

- The deployment diagram portrays the deployment view of the system. It helps in visualizing the topological view of a system. It incorporates nodes, which are physical hardware. The nodes are used to execute the artifacts. The instances of artifacts can be deployed on the instances of nodes.
- Since it plays a critical role during the administrative process, it involves the following parameters:
 1. High performance
 2. Scalability
 3. Maintainability
 4. Portability
 5. Easily understandable
- One of the essential elements of the deployment diagram is the nodes and artifacts. So it is necessary to identify all of the nodes and the relationship between them. It becomes easier to develop a deployment diagram if all of the nodes, artifacts, and their relationship is already known.

○ Example of a Deployment diagram

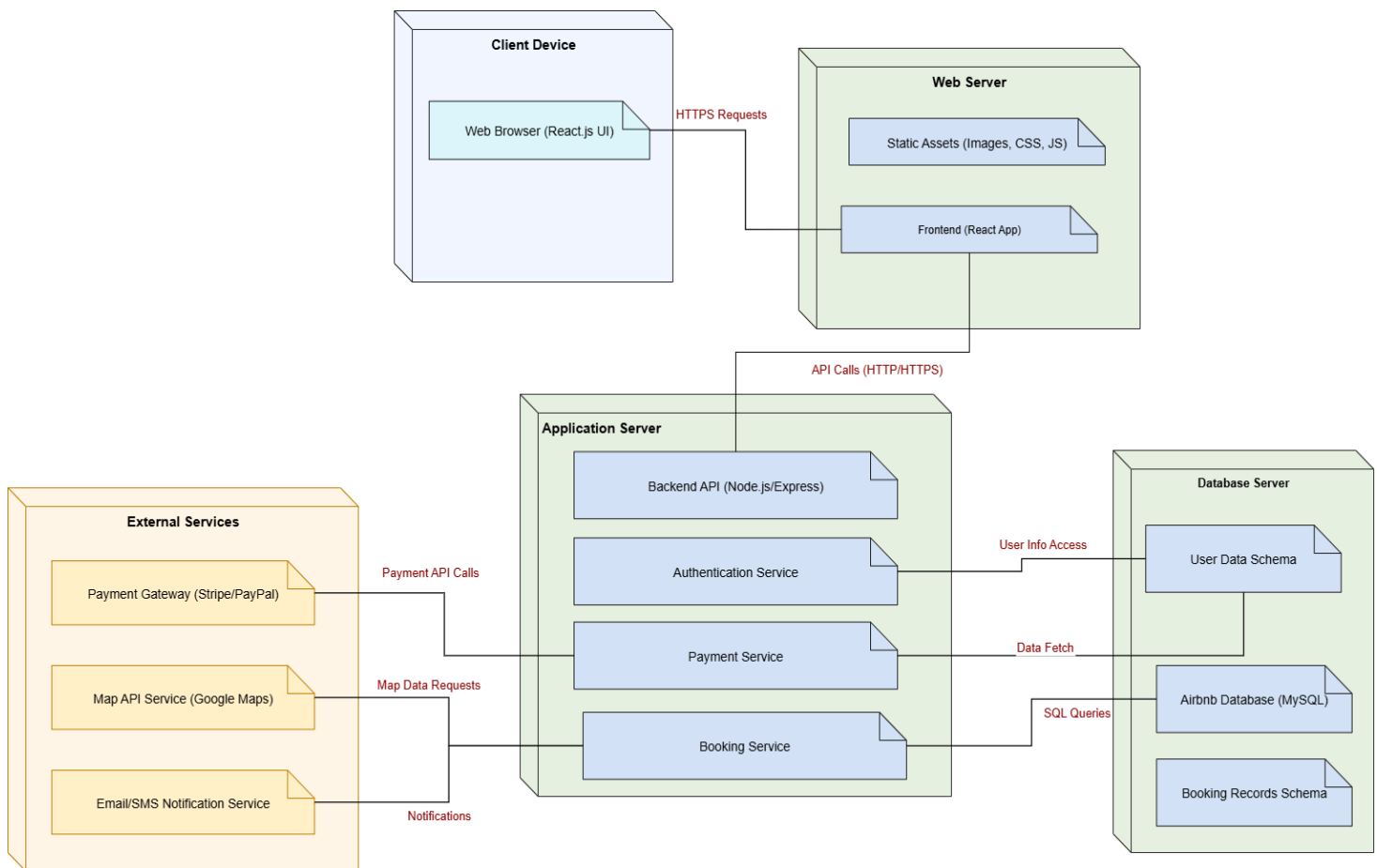
The deployment diagram for the **Airbnb system** illustrates the physical infrastructure and runtime environment supporting the platform's operations. It shows how different system components are deployed across multiple nodes and interact to deliver Airbnb's core functionalities such as property listing, booking, and payment processing.

The system begins with the **Client Device**, where guests and hosts access Airbnb through a web browser or mobile application running the React.js Frontend. All user interactions, including login, browsing available properties, booking accommodations, and making payments, are securely transmitted to the **Web Server** over HTTP/HTTPS connections.

The **Web Server** hosts the **Frontend (React App)** and serves **Static Assets** such as images, CSS, and JavaScript files. It acts as the entry point to the Airbnb platform, ensuring a responsive and interactive user experience. Once a user performs an action, the web server forwards the request to the **Application Server** for processing.

For data persistence, the **Application Server** communicates with the **Database Server** over a secure connection. The **Database Server**, running **MySQL**, stores and manages critical data through artifacts such as the **User Data Schema** and **Booking Records Schema**. These schemas maintain essential information related to user profiles, property listings, and booking histories, ensuring data integrity and consistency across the platform.

Deployment Diagram with Data Flows - Airbnb Room Booking System



The **Application Server** contains the main backend logic of the Airbnb system, developed using **Node.js** and **Express.js**. It includes several key components such as the **Authentication Service** (responsible for user login, registration, and verification), the **Booking Service** (which manages property availability and reservations), and the **Payment Service** (which handles secure payment processing). These components work together to process client requests, enforce business rules, and coordinate data transactions with the database layer.

The system architecture can also be extended to include **External Nodes** such as a **Payment Gateway** (for processing financial transactions securely through services like Stripe or PayPal), an **Email/SMS Notification Service** (for sending booking confirmations and alerts), and a **Map API** (to support property search and location-based services).

This deployment diagram highlights how the client, server, database, and external services interact through well-defined communication paths to deliver a seamless and reliable experience. It provides a clear mapping between Airbnb's software architecture and the physical infrastructure that powers its operations.

Quiz:

- 1) Component diagram and Deployment diagram.
- 2) Draw component diagram to show the run time dependency between a java class file , the java.exe runtime program and the java classes in .zip file.

Suggested Reference:

1. Introduction to the Diagrams of UML 2.X

Rubric wise marks obtained:

Rubrics	1	2	3	4	5	Total
Marks	Complete implementation as asked	Problem analysis	Development of the Solution	Concept Clarity & understanding	Correct answer to all questions	

Signature of Faculty:

Practical – 9

AIM: Prepare the quality management plan.

- **Objectives:** To explore the contents which is required to create quality management plan
- **Theory:**

Quality Management Plan Content

- Overview of Quality Management Plan
- Purpose
- Overview of Plan, Do, Check, Act
- Customer Quality Objectives
- Identify Customer Quality Objectives
- Identify Quality Threshold for each Quality Objective
- Quality Control Plans
- Address each major deliverable
- Identify Independent Technical Review Team(s)
- Quality Assurance
- Organizational Quality System Requirements (Organizational Quality Management Plan)
- Project-specific requirements
- Other Project Specific Information as required

Sample Quality Management Plan

Quality Management Relationships

	Quality Planning	Quality Control (QC)	Quality Assurance (QA)	Quality Improvement
	Plan	Do	Check	Act
What Is Done	Determine what will be quality on the project and how quality will be measured	Monitor specific project products to determine if they meet performance measurement thresholds defined in the quality management plan	Determine if measurement of quality is appropriate by evaluating overall performance on a regular basis to insure the project will satisfy customer quality expectations	Increase the effectiveness and efficiency of the project when corrective actions such as Change Requests are identified. Changes to the Quality Management Plan and the PMP may be required.
When It Is Done	Project Planning Phase Processes • PMP Development • Project Scope & Customer Requirements Definition • Team Establishment • Activity/Schedule Development • Resource Estimate Development • Project Delivery Acquisition Strategy	Project Execution, & Control Phase Processes • Project Execution & Control • Lessons Learned	Project Execution, & Control Phase Processes • Project Execution & Control	Project Execution, & Control Phase and Project Planning Phase Processes • Change Management • PMP Development

Plan

- Identify the customers Quality Objectives. Help customers express quality expectations in objective, quantitative terms.
- Identify professional standards including legal, environmental, economic, code, life safety and health.
- Balance needs and expectations of customers and stakeholders with cost, schedule, and professional standards. Evaluate the costs and benefits of selected quality objectives and the processes to be used to achieve objectives.
- Develop an effective plan and processes, including quality assurance and quality control procedures, to achieve objectives. Consider risk/hazard factors and complexity of the project and adapt processes to provide the requisite level of quality. Document in the risk management plan any project variations from the local QMP requirements.
- Develop performance measure thresholds to ensure agreement on the definition of success relative to Quality Objectives.
- Ensure customer endorsement of all quality objectives included in the Quality Management Plan.

Do

- Do the work according to the approved PMP and standard operating procedures.
- Project execution is a dynamic process. The PDT must communicate, meet on a regular basis, and adapt to changing conditions. The Quality Management Plan and PMP may require modification to ensure that project objectives are met.

Check

- Perform independent technical review, management oversight, and verification to ensure that quality objectives are met consistent with District Quality Management Plans.
- Check performance against the PMP and Customer Quality Objectives performance measures thresholds to verify that performance will accomplish Quality Objectives and to verify sufficiency of the plan. Share findings with all project stakeholders to facilitate continuous improvement.

Act

- If performance measures thresholds are exceeded, take specific corrective actions to fix the systemic cause of any non-conformance, deficiency, or other unwanted effect.
- Document quality improvements that could include appropriate revisions to the quality management plan, alteration of quality assurance and control procedures, and adjustments to resource allocations.

Software Quality Management Plan (SQMP)

Project Title: Airbnb – Online Accommodation Booking System

Version: 1.0

Date: 29-09-2025

Prepared by: Priyanshu Patel, Vishwakarma Government Engineering College

1. Introduction

The Software Quality Management Plan (SQMP) defines the **processes, standards, methods, and responsibilities** to ensure that the Airbnb System meets its defined quality objectives.

It ensures that the system fulfills both:

- **Functional requirements:** User registration, property listing, search, booking, payments, and host management.
- **Non-functional requirements:** Security, scalability, usability, maintainability, and performance as specified in the SRS.

The SQMP integrates organizational quality standards with project-specific requirements and establishes metrics, responsibilities, and review mechanisms to deliver a secure, reliable, and user-friendly accommodation booking platform.

2. Overview of Quality Management Plan

This plan defines the **quality framework** for the Airbnb Booking System. It covers:

- Development quality (processes, standards, reviews).
- Product quality (usability, security, performance, maintainability).
- Assurance activities (audits, reviews, compliance checks).
- Control activities (testing, defect monitoring, metrics evaluation).

Both **process quality** and **product quality** are addressed to achieve a high degree of customer and host satisfaction.

3. Purpose

- Ensure the product meets **functional and non-functional requirements** outlined in the SRS.
- Provide a **roadmap** for achieving high customer satisfaction by delivering a secure, scalable, and reliable booking platform.
- Define **quality standards, metrics, and thresholds** to guide the project from development through deployment.
- Establish **roles, responsibilities, and independent review processes** for quality assurance and control.

4. Overview of Plan–Do–Check–Act (PDCA) Cycle

- **Plan:** Define quality objectives, requirements, and standards (based on SRS and organizational QMS).

- **Do:** Implement coding standards, automated testing, secure design, and documentation.
- **Check:** Conduct quality control activities such as reviews, audits, defect analysis, and performance monitoring.
- **Act:** Apply corrective actions, enhancements, and continuous improvements based on findings.

This cycle ensures continuous quality improvement throughout the project lifecycle.

5. Customer Quality Objectives

5.1 Identify Customer Quality Objectives

1. **Secure and seamless payments** (Stripe/PayPal integration).
2. **Fast and reliable booking process** with <2s response.
3. **High system availability** ($\geq 99.9\%$ uptime).
4. **Intuitive and user-friendly UI/UX** for hosts and guests.
5. **Accurate property availability and booking confirmation.**
6. **Data security and privacy** (encrypted data, HTTPS, PCI-DSS compliance).

5.2 Identify Quality Threshold for Each Objective

Objective	Threshold / Acceptance Criteria
Secure Payments	100% transactions follow PCI-DSS; <1% failure rate
Fast Booking	Average booking response < 2 seconds
System Availability	$\geq 99.9\%$ uptime per month
UI/UX	$\geq 90\%$ positive usability feedback from users
Property Availability Accuracy	0% double-booking cases during testing
Data Security	Zero successful penetration test breaches

6. Quality Standards

The project will adhere to the following standards:

- **ISO/IEC 25010:** Software product quality standards (usability, reliability, security, performance).
- **ISO/IEC 12207:** Software lifecycle processes.
- **OWASP Guidelines:** Secure coding practices.
- **PCI-DSS Compliance:** Payment data protection.
- **Agile Development Best Practices:** Iterative development with frequent quality validation.

7. Quality Assurance (QA) Plan

QA ensures that the processes used to develop Airbnb are effective and meet defined standards.

7.1 Process Assurance

- Requirement reviews for correctness and completeness.
- Code reviews via GitHub Pull Requests.
- Automated CI/CD pipelines for builds and testing.
- Documentation reviews for user and admin manuals.

7.2 Standards Compliance

- Code style: ESLint + Prettier (JavaScript).
- Secure coding: Input validation, encryption for sensitive data.
- Version control: Git with branch management.
- Documentation: Compliance with organizational quality guidelines.

8. Quality Control (QC) Plan

QC ensures the final Airbnb product meets its specifications.

8.1 Testing Strategy

- **Unit Testing:** Validate individual modules (authentication, listing, booking, payment).
- **Integration Testing:** Validate interactions between frontend, backend, and APIs.
- **System Testing:** End-to-end testing of the booking flow.
- **Performance Testing:** Load testing for ≥ 100 concurrent users.
- **Security Testing:** Penetration testing for injection and XSS vulnerabilities.
- **Usability Testing:** Collect user feedback on UI/UX and navigation.

8.2 Tools

- Testing: Jest, Mocha (backend), Cypress (frontend).
- CI/CD: GitHub Actions.
- Performance: Apache JMeter.
- Security: OWASP ZAP.

8.3 Address Each Major Deliverable

- **Frontend:** Responsiveness, accessibility, usability.
- **Backend APIs:** Functionality, integration, and load testing.
- **Database:** Data integrity, consistency, and recovery validation.
- **Payment Module:** Sandbox testing, PCI-DSS compliance validation.
- **Host/Guest Dashboard:** CRUD and reporting verification.
- **Deployment:** CI/CD verification, uptime monitoring.

8.4 Independent Technical Review Teams

- **Code Review Team:** Senior developers not involved in feature creation.
- **QA Team:** Independent testers validating functional/system requirements.
- **Security Review Team:** Internal/external cybersecurity experts for penetration testing.

9. Roles & Responsibilities

- **Project Manager:** Oversees quality objectives and risk management.
- **QA Lead:** Defines QA process and ensures compliance.
- **Developers:** Implement code following standards and write unit tests.
- **Test Engineers:** Execute test cases, log and report defects.
- **Stakeholders:** Participate in acceptance and usability testing.

10. Reviews & Audits

- **Requirement Review:** Check accuracy and completeness of SRS.
- **Design Review:** Validate scalability and data security.
- **Code Review:** Peer review through GitHub PRs.
- **Test Review:** Verify full coverage before deployment.
- **Audit:** Independent quality audit before production release.

11. Organizational Quality System Requirements

- Based on **ISO 9001 Quality Management System (QMS)**.
- Secure coding, documentation, and testing are mandatory.
- Agile methodology with **2-week sprints** for iterative improvements.
- Centralized knowledge base maintained through GitHub Wiki.

12. Project-Specific Requirements

- **Security First:** HTTPS, encryption, PCI-DSS compliance.
- **Scalability:** Support 100 concurrent users, scalable up to 5000+.
- **Backup & Recovery:** Daily automated database backups.
- **Error Handling & Logging:** Centralized monitoring and alerting system.
- **Notifications:** Booking confirmations via Email/SMS API.

13. Other Project-Specific Information

13.1 Tools & Technologies

- **Frontend:** React.js / Next.js (Web), Flutter (Mobile).
- **Backend:** Node.js + Express.js.
- **Database:** MySQL / MongoDB.
- **Testing:** Jest, Mocha, Cypress, JMeter, OWASP ZAP.
- **Deployment:** GitHub Actions, Docker, AWS Cloud Hosting.

13.2 Metrics & Measurements

- **Defect Density:** < 1 per 1000 LOC.
- **Test Coverage:** > 85%.
- **Response Time:** Booking < 2s, Payment < 5s.
- **Reliability:** Downtime < 0.1% monthly.
- **Customer Satisfaction:** ≥ 90% positive user feedback.

13.3 Risk Management

- **Payment Gateway Downtime** → Use alternate providers (PayPal, Stripe).
- **Data Breach** → Enforce strong encryption, regular penetration testing.
- **Traffic Surge** → Apply load balancing, auto-scaling.
- **Schedule Delays** → Agile sprint buffers and milestone tracking.

14. Continuous Improvement

- Conduct sprint retrospectives for lessons learned.
- Maintain a repository of resolved defects and solutions.
- Gather customer feedback post-deployment for improvement.
- Continuously monitor metrics and refine processes.

Quality Management Plan – Summary

	Quality Planning	Quality Control	Quality Assurance	Quality Improvement
	Plan	Do	Check	Act
What is Done	Define customer quality objectives, acceptance criteria, standards (ISO/IEC 25010, PCI-DSS, OWASP), and metrics (uptime,	Perform inspections, reviews, and testing (unit, integration, system, performance, security, usability)	Ensure compliance with defined processes, coding standards, security guidelines, documentation reviews, and process audits.	Collect feedback, conduct retrospectives, track defects, analyze metrics, and refine processes for continuous improvement.

	response time, test coverage).	on each major deliverable.		
When it is Done	Requirement Analysis & Project Planning Phase – Requirement reviews, feasibility analysis, defining acceptance criteria.	Development & Testing Phases – Unit testing, integration testing, usability and performance testing before release.	Entire Project Lifecycle – Requirement review, design validation, code review, continuous integration, compliance audits.	Post-Release & Maintenance Phases – Sprint retrospectives, customer surveys, bug-fix cycles, and continuous improvement activities.

Quiz:

1. Compare Cohesion and coupling with suitable example.
2. Different between software quality assurance vs software quality control.

Suggested Reference:

- 1) Rajib Mall, Fundamentals of software Engineering, Prentice Hall of India.
- 2) Pankaj Jalote, Software Engineering – A Precise Approach Wiley

Rubric wise marks obtained:

Rubrics	1	2	3	4	5	Total
Marks	Complete implementation as asked	Problem analysis	Development of the Solution	Concept Clarity & understanding	Correct answer to all questions	

Signature of Faculty:

Practical – 10

AIM: To perform various testing using the testing tool unit testing, integration testing design test cases..

- **Objectives:** to explore and learn about different testing techniques and use them.

- **Theory:**

- Software Testing is evaluation of the software against requirements gathered from users and system specifications. Testing is conducted at the phase level in software development life cycle or at module level in program code. Software testing comprises of Validation and Verification.

- **Software Validation**

- Validation is process of examining whether or not the software satisfies the user requirements. It is carried out at the end of the SDLC. If the software matches requirements for which it was made, it is validated.
- Validation ensures the product under development is as per the user requirements.
- Validation answers the question – "Are we developing the product which attempts all that user needs from this software ?".
- Validation emphasizes on user requirements.

- **Software Verification**

- Verification is the process of confirming if the software is meeting the business requirements, and is developed adhering to the proper specifications and methodologies.
- Verification ensures the product being developed is according to design specifications.
- Verification answers the question– "Are we developing this product by firmly following all design specifications ?"
- Verifications concentrates on the design and system specifications.

- Target of the test are -

- **Errors** - These are actual coding mistakes made by developers. In addition, there is a difference in output of software and desired output, is considered as an error.
- **Fault** - When error exists fault occurs. A fault, also known as a bug, is a result of an error which can cause system to fail.
- **Failure** - failure is said to be the inability of the system to perform the desired task. Failure occurs when fault exists in the system.

○ Testing Levels

- Testing itself may be defined at various levels of SDLC. The testing process runs parallel to software development. Before jumping on the next stage, a stage is tested, validated and verified.
- Testing separately is done just to make sure that there are no hidden bugs or issues left in the software. Software is tested on various levels -

○ Unit Testing

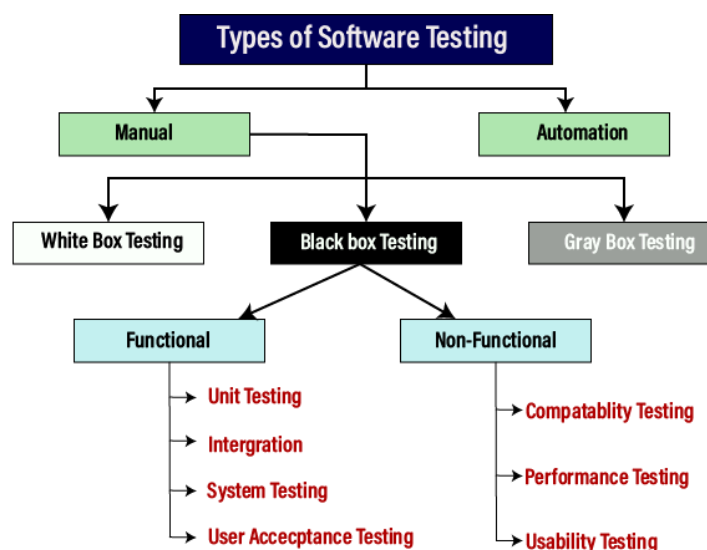
While coding, the programmer performs some tests on that unit of program to know if it is error free. Testing is performed under white-box testing approach. Unit testing helps developers decide that individual units of the program are working as per requirement and are error free.

○ Integration Testing

Even if the units of software are working fine individually, there is a need to find out if the units if integrated together would also work without errors. For example, argument passing and data updation etc.

○ System Testing

The software is compiled as product and then it is tested as a whole.



- **Background / Preparation:**

- **Test management tool**

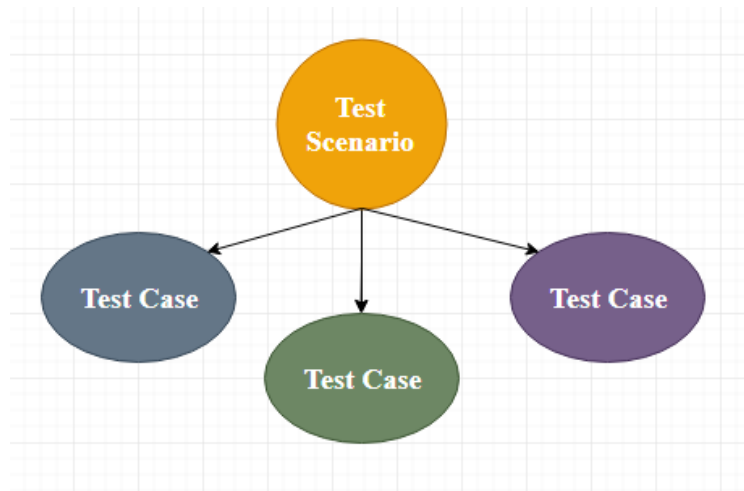
- Test management tools are used to keep track of all the testing activity, fast data analysis, manage manual and automation test cases, various environments, and plan and maintain manual testing as well.
 - Test management tools are used to keep track of all the testing activity, fast data analysis, manage manual and automation test cases, various environments, and plan and maintain manual testing as well.
 - The test management tool is connected with the automation software. These types of tools had various strategies for testing and multiple sets of features. Some of the test management tools had capabilities to design the test case with the help of requirements.
 - It is best for test managing, scheduling, defect logging, tracking, and analysis.
 - Some of the most commonly used test management tools are as follows:
 - Quality center
 - RTH
 - Testpad
 - Test Monitor
 - PractiTest

- **Tools / Material Needed:**

- **Hardware:**
 - **Software:**

- **Procedure / Steps:**

- RTH is another web-based open-source tool, and it stands for **the Requirement and testing hub**. It is used to manage the requirements, test results, and also have bug tracking facilities. This tool follows a structured approach to extend the visibility of the testing process with the help of a common repository for test cases, test result, requirements, and test plans.
 - Test Cases:
 - The test case is defined as a group of conditions under which a tester determines whether a software application is working as per the customer's requirements or not. Test case designing includes preconditions, case name, input conditions, and expected result. A test case is a first level action and derived from test scenarios.



○ Test case template

- The primary purpose of writing a test case is to achieve the efficiency of the application.

Header

Test Case Name/ID :- **Release - Version - Application Name - Module**

Test Case Type:-

F.T.C	I.T.C	S.T.C
-------	-------	-------

Requirement Number:-

Module:-

Severity:- **Critical/Major/Minor**

Status:-

Release:-

Version:-

Pre-condition:-

Test Data:-

Summary:-

Body

Step No.	Description	Inputs	Expected Result	Actual Result	Status	Comments
...	...	...	...	...	...	...
...	...	...	...	...	...	...

Footer

Author:-

Reviewed By:-

Date:-

Approved By:-

As we know, the **actual result** is written after the test case execution, and most of the time, it would be same as the **expected result**. But if the test step will fail, it will be different. So, the actual result field can be skipped, and in **the Comments** section, we can write about the bugs.

And also, the **Input field** can be removed, and this information can be added to the **Description field**.

The above template we discuss above is not the standard one because it can be different for each company and also with each application, which is based on the test engineer and the test lead. But, for testing one application, all the test engineers should follow a usual template, which is formulated.

The test case should be written in simple language so that a new test engineer can also understand and execute the same.

In the above sample template, the header contains the following:

Step number

It is also essential because if step number 20 is failing, we can document the bug report and hence prioritize working and also decide if it's a critical bug.

Test case type

It can be functional, integration or system test cases or positive or negative or positive and negative test cases.

Release

One release can contain many versions of the release.

Pre-condition

These are the necessary conditions that need to be satisfied by every test engineer before starting the test execution process. Or it is the data configuration or the data setup that needs to be created for the testing.

For example: In an application, we are writing test cases to add users, edit users, and delete users. The per-condition will be seen if user A is added before editing it and removing it.

Test data

These are the values or the input we need to create as per the per-condition.

For example, Username, Password, and account number of the users.

The test lead may be given the test data like username or password to test the application, or the test engineer may themselves generate the username and password.

○ *Test Cases – Login Page*

- Following is the possible list of functional and non-functional test cases for a login page:

○ *Functional Test Cases:*

Sr. No.	Functional Test Cases	Type- Negative/ Positive Test Case
1	Verify if a user will be able to login with a valid username and valid password.	Positive
2	Verify if a user cannot login with a valid username and an invalid password.	Negative
3	Verify the login page for both, when the field is blank and Submit button is clicked.	Negative
4	Verify the 'Forgot Password' functionality.	Positive
5	Verify the messages for invalid login.	Positive
6	Verify the 'Remember Me' functionality.	Positive
7	Verify if the data in password field is either visible as asterisk or bullet signs.	Positive
8	Verify if a user is able to login with a new password only after he/she has changed the password.	Positive
9	Verify if the login page allows to log in simultaneously with different credentials in a different browser.	Positive
10	Verify if the 'Enter' key of the keyboard is working correctly on the login page.	Positive
	Other Test Cases	
11	Verify the time taken to log in with a valid username and password.	Performance & Positive Testing
12	Verify if the font, text color, and color coding of the Login page is as per the standard.	UI Testing & Positive Testing
13	Verify if there is a 'Cancel' button available to erase the entered text.	Usability Testing
14	Verify the login page and all its controls in different browsers	Browser Compatibility & Positive Testing.

○ *Non-functional Security Test Cases:*

Sr. No.	Security test cases	Type- Negative/ Positive Test Case
1	Verify if a user cannot enter the characters more than the specified range in each field (Username and Password).	Negative
2	Verify if a user cannot enter the characters more than the specified range in each field (Username and Password).	Positive
3	Verify the login page by pressing 'Back button' of the browser. It should not allow you to enter into the system once you log out.	Negative
4	Verify the timeout functionality of the login session.	Positive
5	Verify if a user should not be allowed to log in with different credentials from the same browser at the same time.	Negative
6	Verify if a user should be able to login with the same credentials in different browsers at the same time.	Positive
7	Verify the Login page against SQL injection attack.	Negative
8	Verify the implementation of SSL certificate.	Positive

Showing 1 to 8 of 8 entries

We can take an **Example** of Gmail Login page. Here is the image of it.

Test Cases for Gmail Login page

Sr. No.	Test Scenarios
1	Enter the valid email address & click next. Verify if the user gets an option to enter the password.
2	Don't enter an email address or phone number & just click the Next button. Verify if the user will get the correct message or if the blank field will get highlighted.
3	Enter the invalid email address & click the Next button. Verify if the user will get the correct message.
4	Enter an invalid phone number & click the Next button. Verify if the user will get the correct message.
5	Verify if a user can log in with a valid email address and password.
6	Verify if a user can log in with a valid phone number and password.
7	Verify if a user cannot log in with a valid phone number and an invalid password.
8	Verify if a user cannot log in with a valid email address and a wrong password.
9	Verify the 'Forgot email' functionality.

Sr. No.	Test Scenarios
10	Verify the 'Forgot password' functionality.

Test Scenarios for the Sign-up page

- 1) Verify the messages for each mandatory field.
- 2) Verify if the user cannot proceed without filling all the mandatory fields.
- 3) Verify the age of the user when the DOB is selected.
- 4) Verify if the numbers and special characters are not allowed in the First and Last name.
- 5) Verify if a user can sign-up successfully with all the mandatory details.
- 6) Verify if a user can log in with the valid details.
- 7) Verify if the Password and Confirm Password fields are accepting similar strings only.
- 8) Verify if the Password field will prompt you for the weak passwords.
- 9) Verify if duplicate email address will not get assigned.
- 10) Verify that hints are provided for each field on the form, for the ease of use.

Test Cases for Airbnb – Online Room Booking System

1. User Registration & Login

Sr. No.	Test Case Description	Type	Expected Result
1	Verify user registration with valid details (Name, Email, Password)	Positive	Registration successful, user account created
2	Verify registration with already registered email	Negative	Error message displayed: "Email already exists"
3	Verify login with valid email & password	Positive	User redirected to homepage/dashboard
4	Verify login with valid email & invalid password	Negative	Error message displayed: "Invalid credentials"
5	Verify login with blank fields and clicking submit	Negative	Validation message shown: "All fields are required"

6	Verify "Forgot Password" functionality	Positive	Password reset link sent to registered email
7	Verify session timeout after inactivity	Security	User session expires, and re-login is required
8	Verify login from multiple devices with same credentials	Positive	Independent sessions created successfully

2. Search & Browse Properties

Sr. No.	Test Case Description	Type	Expected Result
1	Verify browsing all available properties without filters	Positive	Complete property list displayed
2	Verify search by destination, check-in, and check-out dates	Positive	Properties available for entered location/dates shown
3	Verify filter by price range, room type, and amenities	Positive	Correctly filtered property list displayed
4	Verify search with invalid or empty location input	Negative	Message displayed: "No properties found"
5	Verify map view shows accurate property locations	Functional	Map pins correctly represent properties
6	Verify response time for search (≤ 2 seconds)	Performance	Search results displayed within time limit

3. Booking Process

Sr. No.	Test Case Description	Type	Expected Result
1	Verify selecting a property and entering valid booking dates	Positive	Booking summary displayed with payment options
2	Verify booking with invalid dates (check-out < check-in)	Negative	Error message displayed
3	Verify double booking prevention for same property & dates	Negative	Error message shown: "Property unavailable for selected dates"
4	Verify booking history after successful booking	Positive	Booking entry visible under "My Trips"
5	Verify booking cancellation before check-in	Positive	Booking status updated to "Cancelled"
6	Verify refund initiation after cancellation	Positive	Refund initiated as per policy

4. Payment Handling

Sr. No.	Test Case Description	Type	Expected Result
1	Verify payment with valid card/UPI/PayPal details	Positive	Payment successful, confirmation email sent
2	Verify payment with invalid details	Negative	Payment failure with proper error message
3	Verify payment timeout or interruption	Negative	Booking marked as incomplete, retry option shown
4	Verify refund process after booking cancellation	Positive	Refund processed successfully to original payment method

5	Verify secure SSL encryption during payment	Security	All payment data transmitted securely
6	Verify PCI-DSS compliance in payment handling	Security	Compliance validation passed

5. Host Management (Admin/Property Owner Panel)

Sr. No.	Test Case Description	Type	Expected Result
1	Verify host registration and property listing with valid details	Positive	Property successfully added and visible to users
2	Verify updating property details (price, availability, description)	Positive	Property updated successfully
3	Verify deleting a listed property	Positive	Property removed from the platform
4	Verify viewing all bookings for a property	Positive	Booking list displayed correctly
5	Verify unauthorized access to host panel	Security	Access denied, login required
6	Verify host receives notifications for new bookings	Functional	Notification/email sent successfully

6. Non-Functional / Security Test Cases

Sr. No.	Test Case Description	Type	Expected Result
1	Verify user passwords stored in encrypted format in DB	Security	Encrypted, not stored as plain text
2	Verify SQL injection on login and booking forms	Negative	Input rejected, system remains secure
3	Verify system supports 500+ concurrent users	Performance	Application functions without performance degradation
4	Verify cross-browser compatibility (Chrome, Firefox, Safari, Edge)	Compatibility	Application works consistently across browsers
5	Verify mobile responsiveness	Usability	Pages adapt to various screen sizes and devices
6	Verify page load times under 3 seconds	Performance	All key pages load within acceptable time limit
7	Verify accessibility compliance (WCAG 2.1)	Usability	Platform is accessible for all users including those with disabilities

Quiz:

1. What is Software Testing? Explain Black-box and White-Box Testing in details along with examples.
2. What elements of the WebApp can be “unit tested”? What types of tests must be conducted only after the WebApp elements are integrated?
3. What elements of the WebApp can be “unit tested”? What types of tests must be conducted only after the WebApp elements are integrated?
4. What elements of the WebApp can be “unit tested”? What types of tests must be conducted only after the WebApp elements are integrated?
5. What is white box testing? What are the different coverage based testing strategies.
6. What is black box testing? What are the different black box testing techniques?

Suggested Reference:

- 1 Software Testing: A Craftsman's Approach, by Paul C. Jorgensen, Third Edition
- 2 Software Engineering by Rajib Mall, PHI 2014

Rubric wise marks obtained:

Rubrics	1	2	3	4	5	Total
Marks	Complete implementation as asked	Problem analysis	Development of the Solution	Concept Clarity & understanding	Correct answer to all questions	

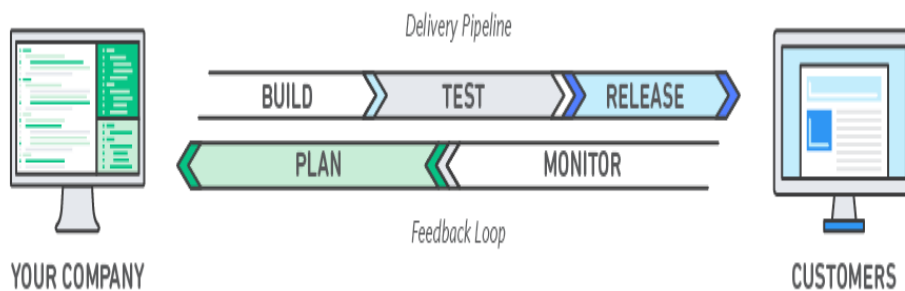
Signature of Faculty:

Practical – 11

AIM: Case Study: Study of Open-source tools in DevOps for Infrastructure Automation, Configuration Management, Deployment Automation, Performance Management, Log Management. Monitoring

- **Objectives:** to explore various case studies where DevOps is applied.
- **Theory:**

DevOps is the combination of cultural philosophies, practices, and tools that increases an organization's ability to deliver applications and services at high velocity: evolving and improving products at a faster pace than organizations using traditional software development and infrastructure management processes. This speed enables organizations to better serve their customers and compete more effectively in the market.



How DevOps Works

Under a DevOps model, development and operations teams are no longer “siloed.” Sometimes, these two teams are merged into a single team where the engineers work across the entire application lifecycle, from development and test to deployment to operations, and develop a range of skills not limited to a single function.

In some DevOps models, quality assurance and security teams may also become more tightly integrated with development and operations and throughout the application lifecycle. When security is the focus of everyone on a DevOps team, this is sometimes referred to as DevSecOps.

These teams use practices to automate processes that historically have been manual and slow. They use a technology stack and tooling which help them operate and evolve applications quickly and reliably. These tools also help engineers independently accomplish tasks (for example, deploying code or provisioning infrastructure) that normally would have required help from other teams, and this further increases a team's velocity.

Why DevOps Matters

Software and the Internet have transformed the world and its industries, from shopping to entertainment to banking. Software no longer merely supports a business; rather it becomes an integral component of every part of a business. Companies interact with their customers through software delivered as online services or applications and on all sorts of devices. They also use software to increase operational efficiencies by transforming every part of the value chain, such as

logistics, communications, and operations. In a similar way that physical goods companies transformed how they design, build, and deliver products using industrial automation throughout the 20th century, companies in today's world must transform how they build and deliver software.

DevOps Practices

Continuous Integration

Continuous integration is a software development practice where developers regularly merge their code changes into a central repository, after which automated builds and tests are run. The key goals of continuous integration are to find and address bugs quicker, improve software quality, and reduce the time it takes to validate and release new software updates.

Continuous Delivery

Continuous delivery is a software development practice where code changes are automatically built, tested, and prepared for a release to production. It expands upon continuous integration by deploying all code changes to a testing environment and/or a production environment after the build stage. When continuous delivery is implemented properly, developers will always have a deployment-ready build artifact that has passed through a standardized test process.

Microservices

The microservices architecture is a design approach to build a single application as a set of small services. Each service runs in its own process and communicates with other services through a well-defined interface using a lightweight mechanism, typically an HTTP-based application programming interface (API). Microservices are built around business capabilities; each service is scoped to a single purpose. You can use different frameworks or programming languages to write microservices and deploy them independently, as a single service, or as a group of services.

Infrastructure as Code

Infrastructure as code is a practice in which infrastructure is provisioned and managed using code and software development techniques, such as version control and continuous integration. The cloud's API-driven model enables developers and system administrators to interact with infrastructure programmatically, and at scale, instead of needing to manually set up and configure resources. Thus, engineers can interface with infrastructure using code-based tools and treat infrastructure in a manner similar to how they treat application code. Because they are defined by code, infrastructure and servers can quickly be deployed using standardized patterns, updated with the latest patches and versions, or duplicated in repeatable ways.

Configuration Management

Developers and system administrators use code to automate operating system and host configuration, operational tasks, and more. The use of code makes configuration changes repeatable and standardized. It frees developers and systems administrators from manually configuring operating systems, system applications, or server software.

Policy as Code

With infrastructure and its configuration codified with the cloud, organizations can monitor and enforce compliance dynamically and at scale. Infrastructure that is described by code can thus be tracked, validated, and reconfigured in an automated way. This makes it easier for organizations to

govern changes over resources and ensure that security measures are properly enforced in a distributed manner (e.g. information security or compliance with PCI-DSS or HIPAA). This allows teams within an organization to move at higher velocity since non-compliant resources can be automatically flagged for further investigation or even automatically brought back into compliance.

Monitoring and Logging

Organizations monitor metrics and logs to see how application and infrastructure performance impacts the experience of their product's end user. By capturing, categorizing, and then analyzing data and logs generated by applications and infrastructure, organizations understand how changes or updates impact users, shedding insights into the root causes of problems or unexpected changes. Active monitoring becomes increasingly important as services must be available 24/7 and as application and infrastructure update frequency increases. Creating alerts or performing real-time analysis of this data also helps organizations more proactively monitor their services.

Communication and Collaboration

Increased communication and collaboration in an organization is one of the key cultural aspects of DevOps. The use of DevOps tooling and automation of the software delivery process establishes collaboration by physically bringing together the workflows and responsibilities of development and operations. Building on top of that, these teams set strong cultural norms around information sharing and facilitating communication through the use of chat applications, issue or project tracking systems, and wikis. This helps speed up communication across developers, operations, and even other teams like marketing or sales, allowing all parts of the organization to align more closely on goals and projects.

DevOps Tools

The DevOps model relies on effective tooling to help teams rapidly and reliably deploy and innovate for their customers. These tools automate manual tasks, help teams manage complex environments at scale, and keep engineers in control of the high velocity that is enabled by DevOps. AWS provides services that are designed for DevOps and that are built first for use with the AWS cloud. These services help you use the DevOps practices described above.

Case Study: Open-source DevOps Tools

1. Infrastructure Automation Tools

These tools allow automatic provisioning and scaling of infrastructure using “Infrastructure as Code (IaC).”

- **Terraform** – Declarative IaC tool, supports multi-cloud (AWS, Azure, GCP).
- **Ansible** – Can also handle infrastructure provisioning with YAML playbooks.
- **Puppet Bolt** – Lightweight orchestration for infrastructure tasks.
- **Vagrant** – Automates VM setup for consistent development environments.

2. Configuration Management Tools

Used to manage and maintain system configurations in a repeatable, error-free manner.

- **Ansible** – Agentless automation for OS configs, package installation, user management.
- **Puppet** – Declarative model for system configuration and compliance.
- **Chef** – Uses recipes (in Ruby DSL) for configuration management at scale.
- **SaltStack** – Event-driven automation and configuration.

3. Deployment Automation Tools

Ensure smooth CI/CD pipelines for fast, reliable deployment.

- **Jenkins** – Popular CI/CD server for automating build, test, deploy.
- **GitLab CI/CD** – Built-in automation pipelines tightly integrated with Git repos.
- **Spinnaker** – Continuous delivery platform supporting multi-cloud deployments.
- **ArgoCD** – Kubernetes-native continuous deployment tool (GitOps-based).

4. Performance Management Tools

Monitor application & server performance, optimize throughput, reduce latency.

- **Apache JMeter** – Open-source tool for load testing web and API performance.
- **Gatling** – High-performance load testing for applications and APIs.
- **k6** – Modern developer-centric performance testing tool.
- **Locust** – Python-based load testing framework for web apps.

5. Log Management Tools

Collect, analyze, and visualize logs for troubleshooting & auditing.

- **ELK Stack (Elasticsearch, Logstash, Kibana)** – Centralized logging, real-time analytics.
- **Graylog** – Log aggregation and search with alerting.
- **Fluentd** – Unified logging layer, often used with Kubernetes.
- **Papertrail (Free tier available)** – Cloud-based log management.

6. Monitoring Tools

Provide visibility into application health, uptime, and infrastructure usage.

- **Prometheus + Grafana** – Metrics collection and visualization (de facto in Kubernetes).
- **Nagios** – Traditional infrastructure monitoring with alerting.

- **Zabbix** – Enterprise-grade monitoring of servers, network, applications.
- **Netdata** – Real-time performance and system monitoring.

+Quiz:

- 1 What are the challenges with DevOps implementation?
- 2 What is DevOps? How it works? What are the DevOps principles & best practices?
- 3 Explain 7Cs of DevOps lifecycle.

Suggested Reference:

- 1 Deepak Gaikwad, Viral Thakkar, DevOps Tools from Practitioner's ViewPoint, Wiley.
- 2 The DevOps Handbook - Gene Kim et. al.

Rubric wise marks obtained:

Rubrics	1	2	3	4	5	Total
Marks	Complete implementation as asked	Problem analysis	Development of the Solution	Concept Clarity & understanding	Correct answer to all questions	

Signature of Faculty: